



```
In [ ]: import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam
```

```
In [ ]: 6. Handwritten Digit Recognition (MNIST Dataset)-Implement a 3-layer neural ne
(Input → Hidden → Output) using Backpropagation.
a. Use Sigmoid, ReLU, or Tanh activation functions.
b. Train the network on the MNIST dataset to classify handwritten digits (0-9)
c. Optimize using Gradient Descent with Momentum or Adam Optimizer.
```

```
In [ ]: (x_train, y_train), (x_test, y_test) = mnist.load_data()
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— 0s 0us/step

```
In [ ]: x_train.shape
```

```
Out[ ]: (60000, 28, 28)
```

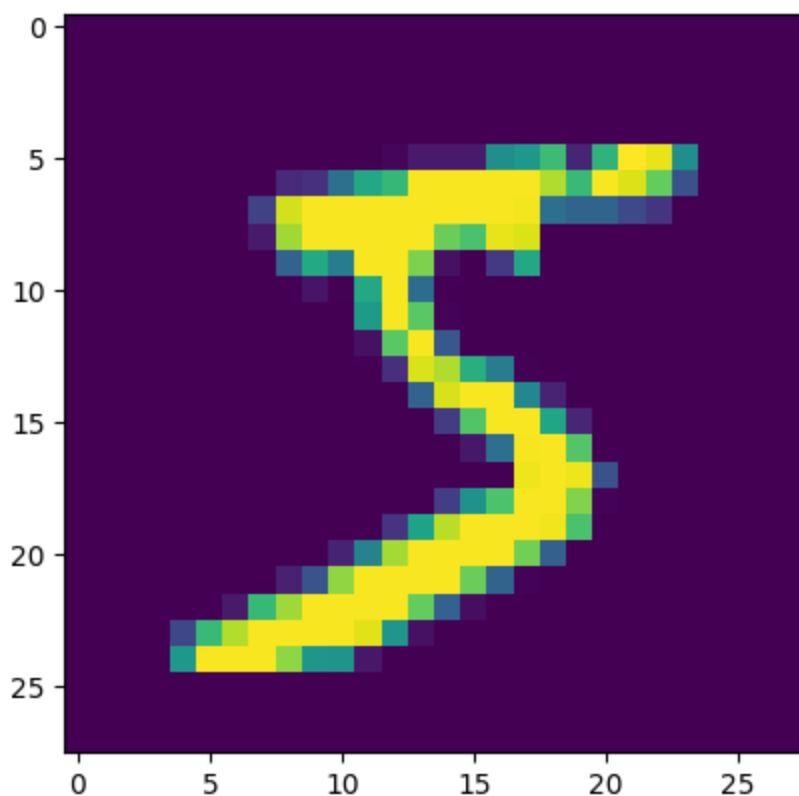
```
In [ ]: x_test.shape
```

```
Out[ ]: (10000, 28, 28)
```

```
In [ ]: x_train = x_train / 255.0
x_test = x_test / 255.0
```

```
In [ ]: import numpy as np
```

```
In [ ]: index=0
plt.imshow(x_train[index])
print(y_train[index])
```



```
In [ ]: modell = Sequential([
    Flatten(input_shape =(28,28)),
    Dense(128, activation = "relu"),
    Dense (10, activation = "softmax")
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:3
7: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. W
hen using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(**kwargs)
```

```
In [ ]: modell.compile(
    optimizer = Adam(),
    loss = 'sparse_categorical_crossentropy',
    metrics = ['accuracy']
)
```

```
In [ ]: history = modell.fit(x_train, y_train, epochs=5, batch_size=32, validation_dat
```

Epoch 1/5
1875/1875 ————— **5s** 2ms/step - accuracy: 0.5905 - loss: 1.6558 -
 val_accuracy: 0.7758 - val_loss: 39.4088
 Epoch 2/5
1875/1875 ————— **4s** 2ms/step - accuracy: 0.8671 - loss: 0.5211 -
 val_accuracy: 0.7858 - val_loss: 51.9125
 Epoch 3/5
1875/1875 ————— **5s** 3ms/step - accuracy: 0.8935 - loss: 0.3917 -
 val_accuracy: 0.7930 - val_loss: 56.4062
 Epoch 4/5
1875/1875 ————— **5s** 2ms/step - accuracy: 0.9044 - loss: 0.3437 -
 val_accuracy: 0.8059 - val_loss: 55.8074
 Epoch 5/5
1875/1875 ————— **6s** 3ms/step - accuracy: 0.9089 - loss: 0.3245 -
 val_accuracy: 0.8172 - val_loss: 54.1168

```
In [ ]: y_prob = model1.predict(x_test)

        y_pred =y_prob.argmax(axis=1)
```

313/313 ————— **0s** 1ms/step

```
In [ ]: from sklearn.metrics import accuracy_score
        accuracy_score(y_test, y_pred)
```

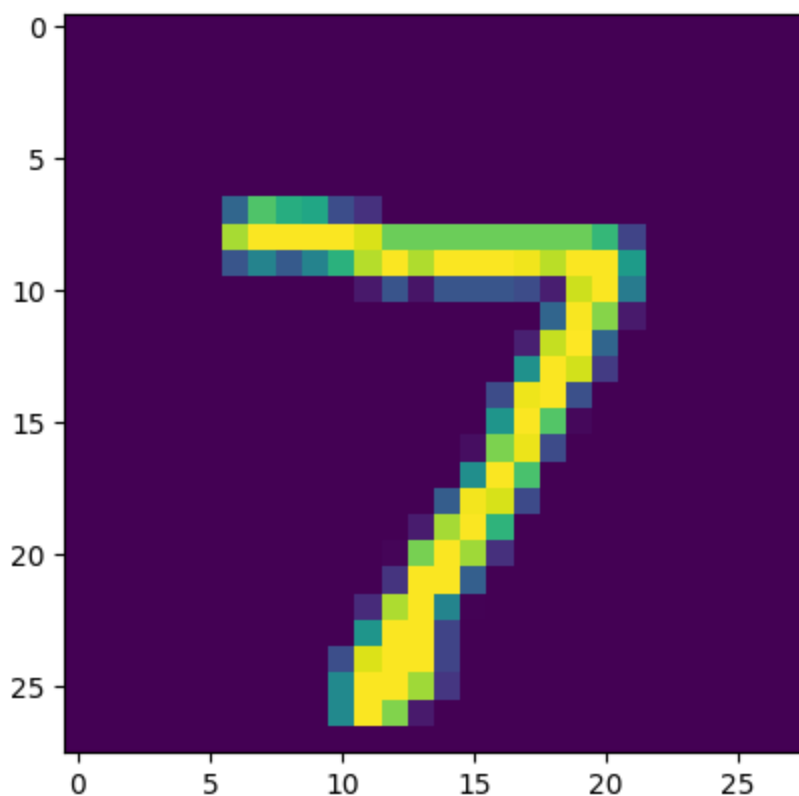
Out[]: 0.8172

```
In [ ]: test_loss, test_acc = model1.evaluate(x_test, y_test)
        print(f"Test Accuracy (Sigmoid): {test_acc:.4f}")
```

313/313 ————— **0s** 1ms/step - accuracy: 0.7977 - loss: 58.6806
 Test Accuracy (Sigmoid): 0.8172

```
In [ ]: plt.imshow(x_test[0])
```

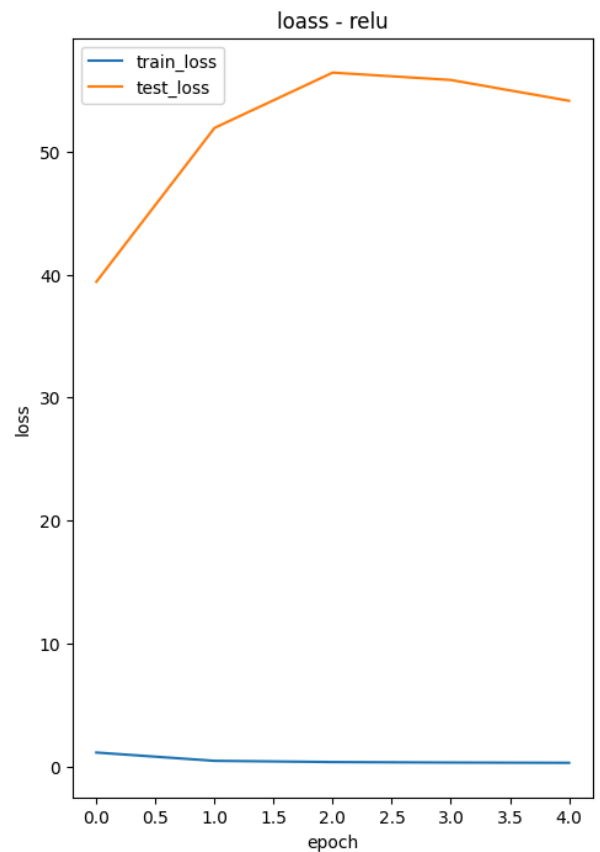
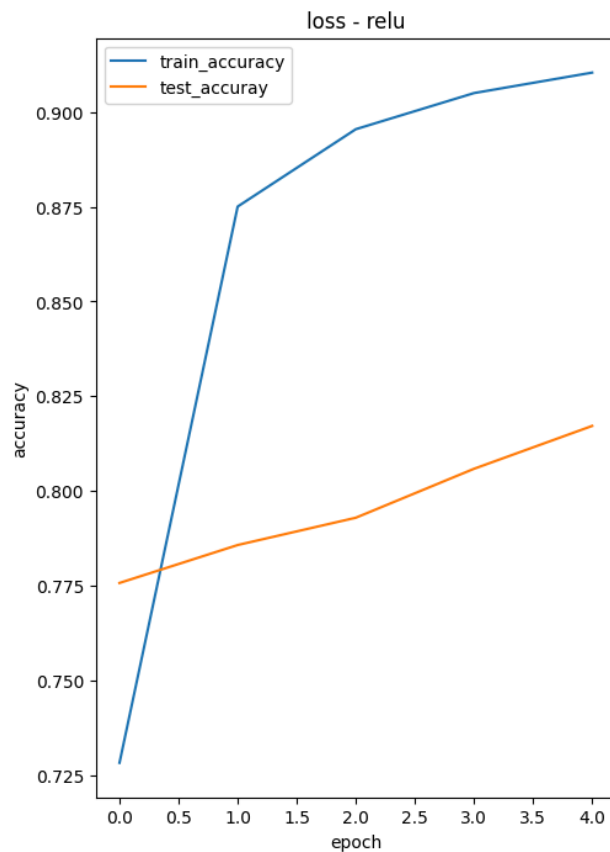
Out[]: <matplotlib.image.AxesImage at 0x7e3323f86120>



```
In [ ]: plt.figure(figsize=(12,8))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label = 'train_accuracy')
plt.plot(history.history['val_accuracy'], label = 'test_accuray')
plt.title('loss - relu')
plt.xlabel('epoch')
plt.ylabel('accuracy')
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label= 'train_loss')
plt.plot(history.history['val_loss'], label = 'test_loss')
plt.title('loass - relu')
plt.xlabel('epoch')
plt.ylabel('loss')
plt.legend()
```

Out[]: <matplotlib.legend.Legend at 0x7e33234284a0>



```
In [ ]: model2 = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='sigmoid'),
    Dense(10, activation='softmax')
])

model2.compile (
    optimizer = Adam(),
    loss = 'sparse_categorical_crossentropy',
    metrics= ['accuracy']
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/resaping/flatten.py:3
7: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. W
hen using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(**kwargs)
```

```
In [ ]: history2= model2.fit(x_train, y_train, epochs =5, batch_size=32, validation_da
```

Epoch 1/5
1875/1875 ————— **4s** 2ms/step - accuracy: 0.8703 - loss: 0.4864 -
 val_accuracy: 0.7864 - val_loss: 1.6750
 Epoch 2/5
1875/1875 ————— **5s** 3ms/step - accuracy: 0.8795 - loss: 0.4427 -
 val_accuracy: 0.7898 - val_loss: 1.7408
 Epoch 3/5
1875/1875 ————— **4s** 2ms/step - accuracy: 0.8880 - loss: 0.4068 -
 val_accuracy: 0.7985 - val_loss: 1.7288
 Epoch 4/5
1875/1875 ————— **5s** 2ms/step - accuracy: 0.8923 - loss: 0.3879 -
 val_accuracy: 0.8070 - val_loss: 1.7010
 Epoch 5/5
1875/1875 ————— **5s** 3ms/step - accuracy: 0.8975 - loss: 0.3684 -
 val_accuracy: 0.8112 - val_loss: 1.7141

```
In [ ]: y_prob1 = model2.predict(x_test)

        y_pred1 = y_prob1.argmax(axis=1)
```

313/313 ————— **0s** 883us/step

```
In [ ]: from sklearn.metrics import accuracy_score
        accuracy_score(y_test, y_pred1)
```

Out[]: 0.8112

```
In [ ]: test_loss, test_acc = model2.evaluate(x_test, y_test)
        print(f"Test Accuracy (Sigmoid): {test_acc:.4f}")
```

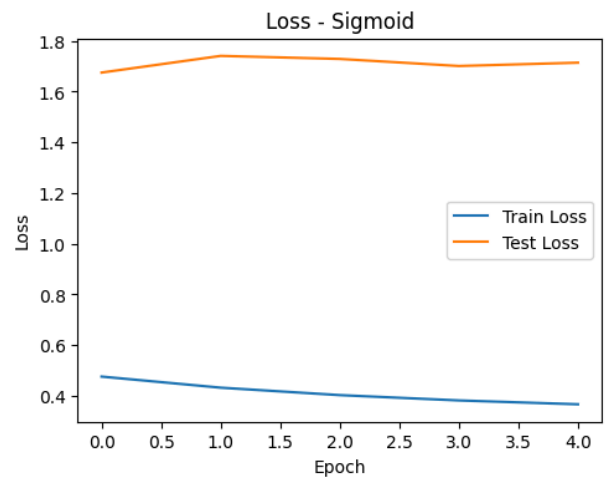
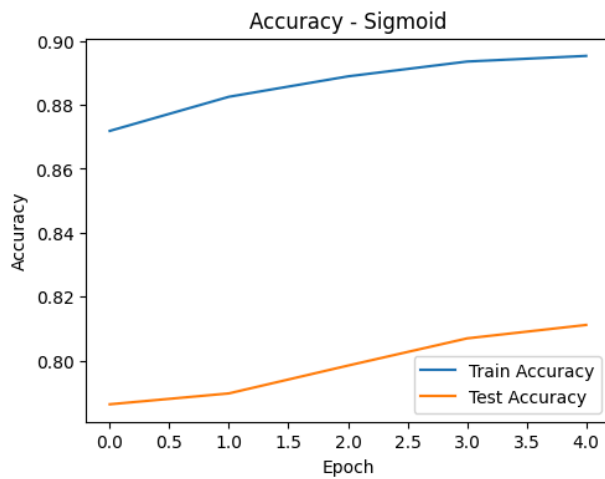
313/313 ————— **0s** 1ms/step - accuracy: 0.7955 - loss: 1.8677
 Test Accuracy (Sigmoid): 0.8112

```
In [ ]: plt.figure(figsize=(12,4))

        plt.subplot(1,2,1)
        plt.plot(history2.history['accuracy'], label='Train Accuracy')
        plt.plot(history2.history['val_accuracy'], label='Test Accuracy')
        plt.title('Accuracy - Sigmoid')
        plt.xlabel('Epoch')
        plt.ylabel('Accuracy')
        plt.legend()

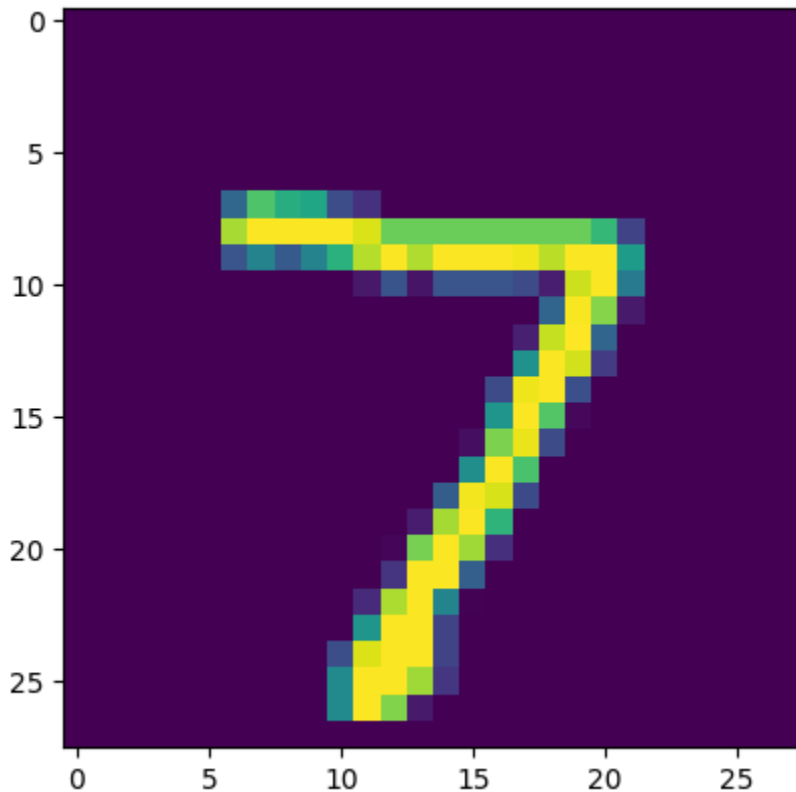
        plt.subplot(1,2,2)
        plt.plot(history2.history['loss'], label='Train Loss')
        plt.plot(history2.history['val_loss'], label='Test Loss')
        plt.title('Loss - Sigmoid')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.legend()

        plt.show()
```



```
In [ ]: plt.imshow(x_test[0])
```

```
Out[ ]: <matplotlib.image.AxesImage at 0x7e3322a750d0>
```



```
In [ ]: model3 = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='tanh'),
    Dense(10, activation='softmax')])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/resaping/flatten.py:3
7: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. W
hen using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
    super().__init__(**kwargs)
```

```
In [ ]: model3.compile(optimizer=Adam(),
                      loss='sparse_categorical_crossentropy',
                      metrics=['accuracy'])
```

```
In [ ]: history3 = model3.fit(x_train, y_train, epochs=5, batch_size=32,
                             validation_data=(x_test, y_test))
```

Epoch 1/5
1875/1875 ————— 5s 2ms/step - accuracy: 0.6230 - loss: 1.4746 -
val_accuracy: 0.7769 - val_loss: 5.2013
Epoch 2/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.8851 - loss: 0.4168 -
val_accuracy: 0.8115 - val_loss: 4.9095
Epoch 3/5
1875/1875 ————— 4s 2ms/step - accuracy: 0.8998 - loss: 0.3436 -
val_accuracy: 0.8321 - val_loss: 4.6205
Epoch 4/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9103 - loss: 0.3108 -
val_accuracy: 0.8506 - val_loss: 4.2530
Epoch 5/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9150 - loss: 0.3012 -
val_accuracy: 0.8498 - val_loss: 4.3950

```
In [ ]: test_loss, test_acc = model3.evaluate(x_test, y_test)
        print(f"Test Accuracy (Tanh): {test_acc:.4f}")
```

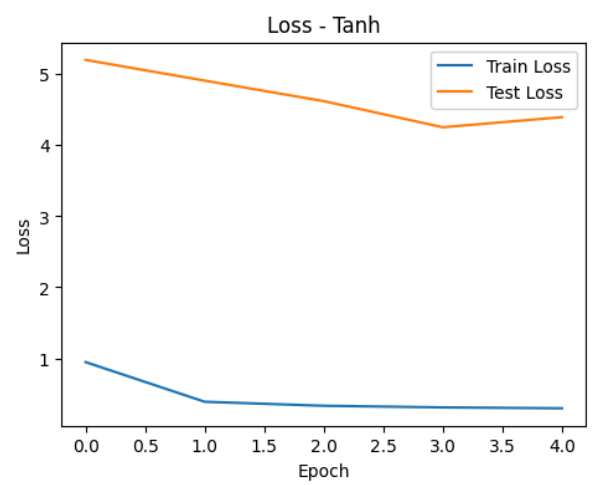
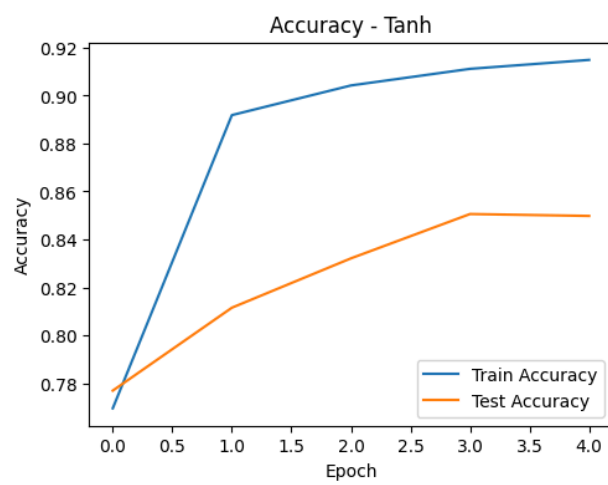
313/313 ————— 1s 2ms/step - accuracy: 0.8288 - loss: 4.7928
Test Accuracy (Tanh): 0.8498

```
In [ ]: plt.figure(figsize=(12,4))

        plt.subplot(1,2,1)
        plt.plot(history3.history['accuracy'], label='Train Accuracy')
        plt.plot(history3.history['val_accuracy'], label='Test Accuracy')
        plt.title('Accuracy - Tanh')
        plt.xlabel('Epoch')
        plt.ylabel('Accuracy')
        plt.legend()

        plt.subplot(1,2,2)
        plt.plot(history3.history['loss'], label='Train Loss')
        plt.plot(history3.history['val_loss'], label='Test Loss')
        plt.title('Loss - Tanh')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.legend()

        plt.show()
```

In []: